Indian E-commerce Returns & Fulfilment Analysis

Data Cleaning and Preparation

1. Import Libraries

```
import pandas as pd
from pathlib import Path
```

2. Load Dataset

```
file = Path("raw_india_ecommerce_orders_returns.csv")

if file.exists():
    data_path = file
else:
    %pip install -q kagglehub
    import kagglehub

    folder = Path(kagglehub.dataset_download(
        "abirambharathisa/india-e-commerce-orders-and-returns-2024-
2026"
    ))
    files = list(folder.rglob("*.csv"))

    if not files:
        raise FileNotFoundError("No CSV file found.")

    data_path = files[0]

df = pd.read_csv(data_path, engine="python")

print("Source file:", data_path)
print("Dataset Shape:", df.shape)
display(df.head())
```

```
Source file: raw_india_ecommerce_orders_returns.csv
Dataset Shape: (165157, 56)
```

```
      order_id  order_item_id       order_datetime customer_id  gender
age  \
0  OD100000001              1  2024-01-01 01:54:00     C015511    Male
33
1  OD100000002              1  2024-01-01 08:04:00     C023908  Female
43
2  OD100000003              1  2024-01-01 08:26:00     C019249    Male
```

```
44
3  OD100000004              1  2024-01-01 08:26:00     C022437  Female
49
4  OD100000004              2  2024-01-01 08:26:00     C022437  Female
49

          city         state region city_tier  ... is_returned
review_rating  \
0  Coimbatore   Tamil Nadu  South    Tier 2  ...       False
NaN
1  Chandigarh   Chandigarh  North    Tier 2  ...       False
NaN
2      Nagpur  Maharashtra   West    Tier 2  ...        True
NaN
3        Pune  Maharashtra   West    Tier 1  ...       False
4.0
4        Pune  Maharashtra   West    Tier 1  ...       False
NaN

  return_id return_requested_date days_to_return
return_reason  \
0       NaN                   NaN            NaN
NaN
1       NaN                   NaN            NaN
NaN
2  RT500001            2024-01-08            4.0  Quality not as
expected
3       NaN                   NaN            NaN
NaN
4       NaN                   NaN            NaN
NaN

   return_status    refund_mode  refund_amount  refund_days
0            NaN            NaN            NaN          NaN
1            NaN            NaN            NaN          NaN
2       Approved  Bank Transfer          709.0          8.0
3            NaN            NaN            NaN          NaN
4            NaN            NaN            NaN          NaN

[5 rows x 56 columns]
```

3. Clean Column Names and Text

```
df.columns = (
    df.columns.str.strip()
    .str.lower()
    .str.replace(r"[^a-z0-9]+", "_", regex=True)
    .str.strip("_")
)
```

```
for col in df.select_dtypes(include=["object","str"]).columns:
    df[col] = df[col].str.strip()
```

4. Select Relevant Columns

```
keep_cols = [
    "order_id", "order_item_id", "order_datetime", "customer_id",
    "state", "region", "is_plus_member", "payment_method",
    "category", "sub_category", "brand", "product_id", "seller_id",
    "seller_rating", "quantity", "mrp", "discount_pct",
"selling_price",
    "item_total", "courier_partner", "promised_delivery_days",
    "shipped_datetime", "delivered_datetime", "actual_delivery_days",
    "is_late_delivery", "order_status", "cancel_reason",
"item_status",
    "is_returned", "return_reason", "return_status", "refund_mode",
    "refund_amount", "refund_days"
]

missing_cols = [col for col in keep_cols if col not in df.columns]

if missing_cols:
    raise KeyError(f"Missing columns: {missing_cols}")

df = df[keep_cols].copy()
```

5. Fix Data Types and Remove Duplicates

```
date_cols = [
    "order_datetime", "shipped_datetime", "delivered_datetime"
]

for col in date_cols:
    df[col] = pd.to_datetime(df[col], errors="coerce")

numeric_cols = [
    "seller_rating", "quantity", "mrp", "discount_pct",
    "selling_price", "item_total", "promised_delivery_days",
    "actual_delivery_days", "is_late_delivery",
    "refund_amount", "refund_days"
]

for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors="coerce")

for col in ["is_plus_member", "is_returned"]:
    values = df[col].astype("string").str.strip().str.lower()
    df[col] = values.map({"true": 1, "false": 0, "1": 1, "0": 0})

df = df.drop_duplicates().reset_index(drop=True)
```

```
print("Cleaned Dataset Shape:", df.shape)
```

```
Cleaned Dataset Shape: (165157, 34)
```

6. Check Missing Values and Data Quality

```
missing = pd.DataFrame({
    "Missing Count": df.isna().sum(),
    "Missing Percentage": (df.isna().mean() * 100).round(2)
})

display(missing[missing["Missing Count"] > 0])

checks = pd.Series({
    "Missing Order IDs": df["order_id"].isna().sum(),
    "Missing Order-Item IDs": df["order_item_id"].isna().sum(),
    "Duplicate Order-Item Keys": df.duplicated(
        ["order_id", "order_item_id"]
    ).sum(),
    "Non-positive Quantities": (df["quantity"] <= 0).sum(),
    "Invalid Discounts": (
        (df["discount_pct"] < 0) | (df["discount_pct"] > 100)
    ).sum()
})

display(checks.to_frame("Result"))
```

	Missing Count	Missing Percentage
shipped_datetime	7016	4.25
delivered_datetime	8853	5.36
actual_delivery_days	8853	5.36
is_late_delivery	8853	5.36
cancel_reason	158141	95.75
return_reason	144314	87.38
return_status	144314	87.38
refund_mode	144314	87.38
refund_amount	144314	87.38
refund_days	145386	88.03

	Result
Missing Order IDs	0
Missing Order-Item IDs	0
Duplicate Order-Item Keys	0
Non-positive Quantities	0
Invalid Discounts	0

7. Export Cleaned Dataset

```
output_file = Path("Cleaned_Data.csv")
df.to_csv(output_file, index=False)

print("Cleaned dataset saved:", output_file.resolve())
print("Final Shape:", df.shape)
```

```
Cleaned dataset saved: C:\Users\khush\Downloads\Cleaned_Data.csv
Final Shape: (165157, 34)
```